

Part 1

The dataset contains 7,043 rows and 21 columns, including features related to customer demographics, services, and billing information, with **Churn** as the target variable indicating whether a customer left. Most columns do not have missing values, but **TotalCharges** has some due to being stored as text. The dataset is imbalanced, with about 73% of customers staying and 27% leaving. This is a classification problem, and it is important in business because identifying customers likely to churn can help improve retention and reduce customer loss.

Part 2

To prepare the dataset, I separated features into numeric and categorical variables, converted **TotalCharges** to numeric and filled missing values using the median, and transformed the target variable **Churn** into binary form. I used one-hot encoding for categorical features and split the data into training and testing sets using stratification to handle the imbalanced dataset.

Part 3

A Logistic Regression model was used as the baseline model. It was trained using a preprocessing pipeline and evaluated using multiple metrics.

Performance:

- Accuracy: 0.804
- Precision: 0.654
- Recall: 0.556
- F1-score: 0.601
- ROC-AUC: 0.843

The baseline model shows stable performance and provides a good reference for comparison.

Part 4

I tuned `n_estimators`, `max_depth`, `learning_rate`, and `subsample` using `GridSearchCV`. These parameters were chosen because they control model complexity, learning speed, and help prevent overfitting.

Part 5

I chose F1-score because it balances precision and recall, which is important for churn prediction. It is more informative than accuracy since the dataset is imbalanced, and accuracy alone may not properly reflect how well the model identifies churn customers.

Part 6

Using the trained XGBoost model, I identified the top important features, including contract type, monthly charges, and tenure. Customers with short-term contracts and higher monthly charges are more likely to churn, while customers with longer tenure tend to stay. This suggests that customers who are less committed or paying more are at higher risk of leaving.

Part 7

I think XGBoost performed slightly better overall, especially in accuracy, precision, and ROC-AUC, while Logistic Regression did better in recall and F1-score. XGBoost works better because it can capture more complex patterns, but it is harder to interpret.

Result

XGBoost performed slightly better overall, especially in accuracy, precision, and ROC-AUC. However, Logistic Regression performed better in recall and F1-score, meaning it was better at identifying churn customers.

Summary

The dataset was explored and cleaned, missing values were handled, and numeric and categorical features were separated. Preprocessing pipelines were applied, a Logistic Regression model was trained as a baseline, and an XGBoost model was built with hyperparameter tuning using GridSearchCV. Finally, both models were evaluated and compared using multiple metrics.

Code

Data Preprocessing

```
df_clean = df.copy()
df_clean = df_clean.drop(columns=["customerID"])
df_clean["Churn"] = df_clean["Churn"].map({"Yes": 1, "No": 0})

df_clean["TotalCharges"] = pd.to_numeric(df_clean["TotalCharges"],
errors="coerce")
df_clean["TotalCharges"] =
df_clean["TotalCharges"].fillna(df_clean["TotalCharges"].median())
```

Train/Test Split

```
X = df_clean.drop(columns=["Churn"])
y = df_clean["Churn"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Baseline Model

```
baseline_model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=2000))
])
```

```
baseline_model.fit(X_train, y_train)
baseline_preds = baseline_model.predict(X_test)
```